

JAVA ADVANCED HASHMAP - JVM - CONCURRENCY - STREAMS

Deep-dive Q&A; for backend interviews | 34 questions across 10 areas

HashMap & Hashing Internals	5 questions
ConcurrentHashMap & Thread-Safe Collections	4 questions
JVM Memory Model	3 questions
Garbage Collection	4 questions
Multithreading & Thread Lifecycle	3 questions
Synchronization, volatile & Locks	3 questions
Java 8 Streams API	3 questions
Lambdas & Functional Interfaces	3 questions
CompletableFuture & Async Programming	3 questions
Best Practices & Common Mistakes	3 questions

Prepared for: CJDC - Creative Java Developers Community
Slogan: Grow Together

Contents

1. HashMap & Hashing Internals	5 questions
2. ConcurrentHashMap & Thread-Safe Collections	4 questions
3. JVM Memory Model	3 questions
4. Garbage Collection	4 questions
5. Multithreading & Thread Lifecycle	3 questions
6. Synchronization, volatile & Locks	3 questions
7. Java 8 Streams API	3 questions
8. Lambdas & Functional Interfaces	3 questions
9. CompletableFuture & Async Programming	3 questions
10. Best Practices & Common Mistakes	3 questions

How to use this guide

- For every internals question (HashMap, GC, ConcurrentHashMap), be ready to draw the data structure.
- Interviewers often follow up with 'why' - know the reasoning behind a design choice, not just the fact.
- Practice the volatile vs synchronized vs ReentrantLock comparison out loud - it comes up constantly.
- Tie each concept back to a real bug or decision from your own project experience when you can.

Q1. How does HashMap store and retrieve entries internally?

HashMap is backed by an array of buckets (default size 16). On `put()`, it computes `key.hashCode()`, runs it through an internal spreading function to reduce collisions, and uses $(n-1) \& \text{hash}$ to pick a bucket index. If the bucket is empty, the entry is placed directly; if occupied, it's appended to that bucket's chain after an `equals()` check to see if the key already exists (in which case the value is replaced).

Q2. What happens on a hash collision, and why does Java 8 introduce tree buckets?

Before Java 8, colliding entries in the same bucket formed a linked list, so worst-case lookup in a badly-collided bucket was $O(n)$. Java 8 automatically converts a bucket's linked list into a red-black tree once it exceeds 8 entries (and the table is large enough), bringing worst-case lookup down to $O(\log n)$ - this specifically defends against maliciously crafted keys designed to collide and degrade performance.

Q3. Why must `equals()` and `hashCode()` be overridden together?

The contract requires that if two objects are equal per `equals()`, they must return the same `hashCode()`. Overriding only `equals()` breaks this: two 'equal' objects could hash to different buckets, so a HashMap or HashSet would treat them as distinct entries and lookups would silently fail to find an object you know is logically present.

Q4. What triggers a HashMap resize, and what does it cost?

HashMap resizes (doubles its bucket array) once the number of entries exceeds `capacity * load factor` (default load factor 0.75, so a default 16-bucket map resizes after 12 entries). Resizing requires rehashing and redistributing every existing entry into the new, larger array, which is an $O(n)$ operation - this is why pre-sizing a HashMap when you know the expected entry count avoids repeated resize costs.

Q5. Why doesn't ConcurrentHashMap allow null keys or values, while HashMap does?

In a single-threaded HashMap, `get()` returning null is unambiguous - it means 'key not found or the value is actually null,' and you can disambiguate with `containsKey()`. In a concurrently accessed map, that ambiguity becomes dangerous: another thread could be removing the key at the exact moment you call `get()`, so null could mean several different things at once. Disallowing null entirely removes that race condition.

Q1. Why is a plain HashMap unsafe under concurrent modification?

HashMap has no internal synchronization. Concurrent structural modification (e.g., two threads triggering a resize at once) can corrupt its internal linked list structure, and even simple concurrent reads/writes can throw a `ConcurrentModificationException` if an iterator detects the map changed underneath it (its fail-fast behavior).

Q2. How does ConcurrentHashMap achieve thread safety without locking the entire map?

Since Java 8, it uses fine-grained locking: writes synchronize only on the head node of the specific bucket being modified, and use CAS (compare-and-swap) operations for lock-free insertion into an empty bucket. Reads are largely lock-free. This means two threads writing to different buckets don't block each other at all, unlike a `Collections.synchronizedMap()` wrapper which locks the whole map on every operation.

Q3. What is the difference between ConcurrentHashMap and Collections.synchronizedMap()?

`synchronizedMap()` wraps a regular HashMap and synchronizes every single method call on one global lock, so only one thread can touch the map at a time - a bottleneck under high concurrency. `ConcurrentHashMap` uses per-bucket locking, allowing many threads to read and write concurrently as long as they're not contending for the exact same bucket, giving far higher throughput.

Q4. Is a compound operation like 'check-then-put' safe on ConcurrentHashMap?

Not if done as two separate calls (`containsKey()` then `put()`) - another thread could insert between the two calls, causing a lost update. `ConcurrentHashMap` provides atomic compound methods for exactly this - `putIfAbsent()`, `computeIfAbsent()`, `merge()` - which perform the check-and-act as a single atomic operation.

Q1. What are the main memory areas the JVM divides at runtime?

Heap (shared across all threads, stores objects and is garbage collected), Stack (one per thread, holds local variables, method parameters, and call frames - LIFO and thread-private), Metaspace (stores class metadata, replacing the old PermGen, and grows dynamically using native memory), and the PC Register plus Native Method Stack (used for bytecode execution bookkeeping and native/JNI calls).

Q2. What's the practical difference between a `StackOverflowError` and an `OutOfMemoryError`?

`StackOverflowError` happens when a thread's call stack exceeds its allocated size - almost always caused by excessive or unterminated recursion. `OutOfMemoryError` (heap space) happens when the heap is full of live, reachable objects and the garbage collector can't free enough space to satisfy a new allocation - usually a sign of a memory leak or genuinely needing more heap.

Q3. How is the heap subdivided, and why does that subdivision exist?

The heap is split into a Young Generation (further divided into Eden and two Survivor spaces) and an Old Generation. Most objects are short-lived, so new objects are allocated in Eden and collected frequently and cheaply (Minor GC); objects that survive several Minor GCs are promoted ('tenured') into the Old Generation, which is collected less often but more expensively. This generational split exploits the observation that most garbage dies young.

Q1. What is the difference between a Minor GC, a Major GC, and a Full GC?

A Minor GC cleans only the Young Generation and is fast and frequent. A Major GC cleans the Old Generation. A Full GC cleans the entire heap (Young + Old, and often Metaspace too) and is the most expensive, typically causing the longest application pause - minimizing Full GC frequency is a common tuning goal.

Q2. Compare G1 and ZGC for a low-latency service.

G1 (Garbage First) divides the heap into many equally-sized regions and prioritizes collecting the regions with the most garbage first, targeting a configurable pause-time goal, typically achieving pauses in the tens of milliseconds. ZGC is designed for much larger heaps and sub-millisecond pause times using concurrent, colored-pointer-based collection that does almost all of its work without stopping application threads - the right choice when your SLA genuinely can't tolerate G1-level pauses, at the cost of somewhat higher CPU overhead.

Q3. What does 'stop-the-world' mean, and why can't GC avoid it entirely?

It means the JVM pauses all application threads so the collector can safely inspect and move objects without the program mutating references mid-collection. Fully concurrent collectors like ZGC or Shenandoah minimize these pauses to very short synchronization points, but some brief stop-the-world phase is still typically needed to keep object graph traversal correct under concurrent mutation.

Q4. What are common JVM flags used to tune garbage collection?

-Xms and -Xmx set the initial and maximum heap size (setting them equal avoids costly heap resizing at runtime). -XX:+UseG1GC selects the G1 collector. -XX:MaxGCPauseMillis sets a target pause time goal for G1 to aim for. Beyond flags, GC logs and tools like VisualVM or async-profiler are used to actually measure whether tuning improved anything, rather than tuning blind.

Q1. What are the states in a Java thread's lifecycle?

NEW (created, not yet started), RUNNABLE (eligible to run, whether actually executing or waiting for CPU time), BLOCKED (waiting to acquire a monitor lock held by another thread), WAITING (waiting indefinitely via `wait()`/`join()` with no timeout), TIMED_WAITING (waiting with a timeout via `sleep()` or a timed `wait()`), and TERMINATED (finished execution).

Q2. What's the difference between implementing Runnable and Callable?

Runnable's `run()` method returns no value and cannot throw a checked exception. Callable's `call()` method returns a value (wrapped in a `Future` when submitted to an `ExecutorService`) and can throw checked exceptions, which the caller retrieves when calling `Future.get()`. Use Callable whenever the task needs to produce a result or propagate a checked exception.

Q3. Why prefer an ExecutorService/thread pool over manually creating new Thread objects?

Creating a new OS thread per task is expensive and unbounded thread creation under load can exhaust memory and crash the JVM with an `OutOfMemoryError`. A thread pool reuses a fixed or bounded set of worker threads, applies backpressure via a task queue, and centralizes lifecycle management (shutdown, exception handling) instead of scattering ad-hoc Thread creation throughout the codebase.

Q1. What does volatile actually guarantee, and what does it not guarantee?

volatile guarantees visibility - a write by one thread is immediately visible to other threads reading that field, and it prevents certain compiler/CPU reorderings around the field. It does NOT guarantee atomicity - a compound operation like `count++` on a volatile `int` is still a read-modify-write sequence that two threads can interleave, causing a lost update, which is why `AtomicInteger` exists for atomic increments.

Q2. What's the difference between synchronized and a ReentrantLock?

`synchronized` is a built-in, implicit monitor lock acquired and released automatically at block/method boundaries - simple but inflexible. `ReentrantLock` is an explicit lock from `java.util.concurrent.locks` requiring manual `lock()/unlock()` (always in a `try/finally`), but offers `tryLock()` with a timeout, `lockInterruptibly()` to respond to interruption while waiting, and fairness ordering - useful when you need more control than `synchronized` provides.

Q3. What are the four necessary conditions for a deadlock, and how do you avoid it?

Mutual exclusion (a resource can't be shared), hold-and-wait (a thread holds one lock while waiting for another), no preemption (a lock can't be forcibly taken away), and circular wait (a cycle of threads each waiting on a lock the next one holds). The most practical fix is breaking circular wait - always acquire multiple locks in the same global order across every thread - or using `tryLock()` with a timeout so a thread can back off instead of waiting forever.

Q1. What is the difference between an intermediate and a terminal stream operation?

Intermediate operations (filter, map, sorted, distinct, limit) are lazy - they just build up a pipeline description and return a new Stream without doing any work. Terminal operations (collect, forEach, reduce, count) actually trigger execution of the whole pipeline and produce a result or side effect; without a terminal operation, nothing in the stream ever runs.

Q2. Why are streams lazy, and what's the practical benefit?

Laziness lets the JVM fuse multiple intermediate operations into a single pass over the data and short-circuit early when possible - for example, filter().findFirst() can stop as soon as one match is found instead of processing the entire source collection, which a naive eager implementation couldn't do.

Q3. When is a parallel stream actually worth using?

Parallel streams help mainly with large datasets and CPU-bound, independent per-element work, since they split work across the common ForkJoinPool. For small collections, the overhead of splitting and merging work across threads usually costs more than it saves - a plain sequential stream or even a for-loop is faster. Parallel streams also assume operations are stateless and non-interfering; using shared mutable state inside them causes race conditions.

Q1. What is a functional interface, and how does `@FunctionalInterface` help?

A functional interface has exactly one abstract method (a 'SAM' - single abstract method), which is what a lambda expression implements. `@FunctionalInterface` isn't required for this to work, but it makes the compiler enforce that the interface still has exactly one abstract method, catching accidental changes that would break lambda compatibility.

Q2. What's the difference between Supplier, Consumer, Function, and Predicate?

Supplier takes nothing and produces a T (`get()`) - used to lazily produce a value. Consumer takes a T and returns nothing (`accept()`) - used for side effects like `forEach()`. Function takes a T and returns an R (`apply()`) - used for transformations like `map()`. Predicate takes a T and returns a boolean (`test()`) - used for filtering conditions.

Q3. What's the advantage of a method reference like `String::length` over an equivalent lambda?

It's purely a readability/conciseness improvement - `s -> s.length()` and `String::length` compile to functionally identical bytecode. Method references communicate intent slightly more directly when you're just forwarding to an existing method with no extra logic.

Q1. What's the key limitation of the plain Future interface that CompletableFuture fixes?

A plain Future only supports blocking on `get()` to retrieve a result - there's no way to attach a callback, chain further async work, or combine multiple futures together. `CompletableFuture` supports non-blocking composition: `thenApply()` to transform a result, `thenAccept()` to consume it, `thenCombine()` to merge two independent futures, and `exceptionally()/handle()` to deal with failures, all without blocking a thread on `get()`.

Q2. How do you run several independent async calls and wait for all of them to finish?

`CompletableFuture.allOf(future1, future2, future3)` returns a `CompletableFuture` that completes once every supplied future completes, letting you fire off independent I/O calls in parallel and join once they're all done rather than awaiting them sequentially.

Q3. What thread pool does `supplyAsync()` run on by default, and why does that matter?

By default it runs on the common `ForkJoinPool`, which is shared JVM-wide - including by parallel streams. For CPU-light but I/O-heavy async work (like HTTP calls), it's common practice to supply a dedicated `Executor` to `supplyAsync()` instead, so slow I/O-bound tasks don't starve the shared pool that other parts of the application depend on.

Q1. What's a common mistake with HashMap keys and mutable objects?

Using a mutable object as a HashMap key and then mutating a field that's part of its hashCode() after insertion. The object's hash changes, but the map doesn't rehash it into the new bucket, so future lookups with an 'equal' key silently fail to find the entry - a classic and hard-to-debug bug. Prefer immutable keys (String, Integer, or your own immutable value objects/records).

Q2. Why is blocking on CompletableFuture.get() in the middle of otherwise-async code a mistake?

It defeats the entire purpose of using CompletableFuture - you're back to tying up a thread waiting synchronously, which under load causes the same thread-pool exhaustion problems async code was meant to avoid. If you must bridge into blocking code, do it at the very edge of the call chain, not buried inside composed async logic.

Q3. When should you return Optional instead of null?

Optional makes the possibility of 'no value' explicit and type-safe as a return type from a method, forcing the caller to consciously handle the empty case instead of risking an unchecked NullPointerException. It's best used for method return types, not for fields, method parameters, or collection elements, where it adds overhead without much benefit.